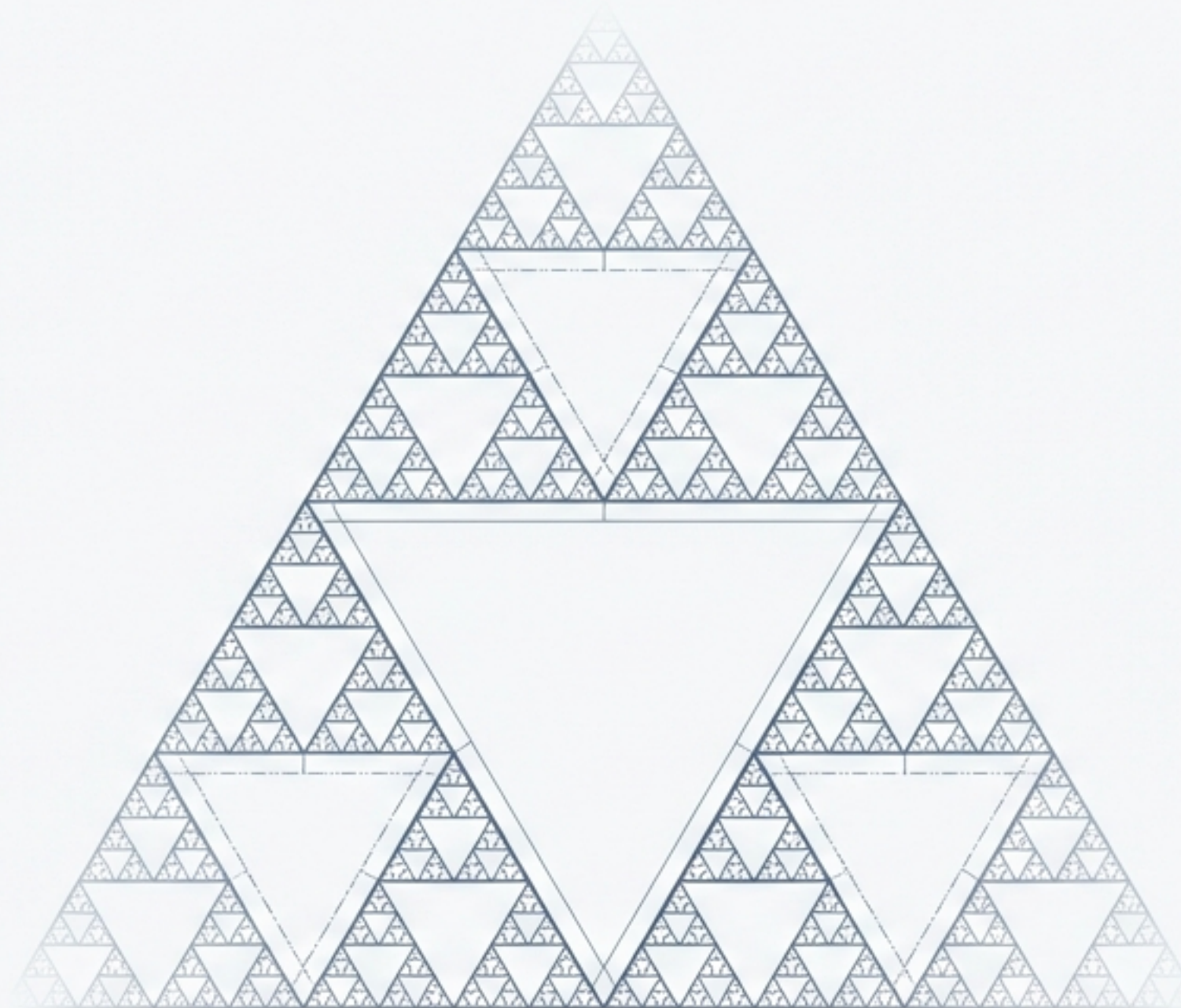


Recursion in Python

Mastering the Art of Functions Calling Themselves



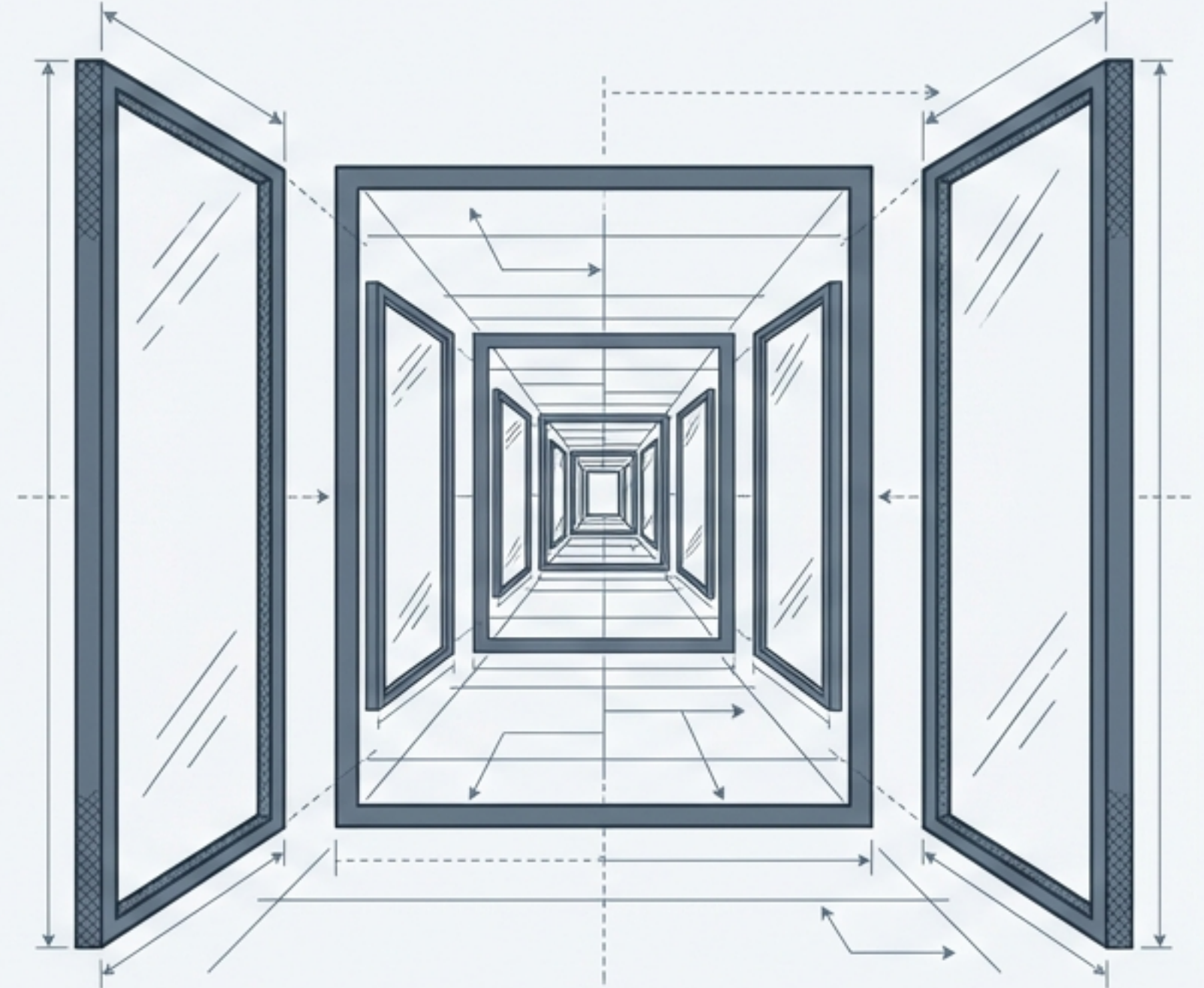
Based on the Chai aur Code Lecture Series

The Core Definition

Recursion is a function calling itself.

In standard programming, we call a function from the outside. In recursion, we call the function from within its own body to solve a repeated task.

“Like the movie ‘Kartik calling Kartik’—when you call yourself, that is recursion.”



Iteration vs. Recursion

Iteration (Loops)

Inter Tight



- Uses 'for' or 'while' loops.
- Great for simple repetition.
- Code can become verbose for complex logic.

Recursion

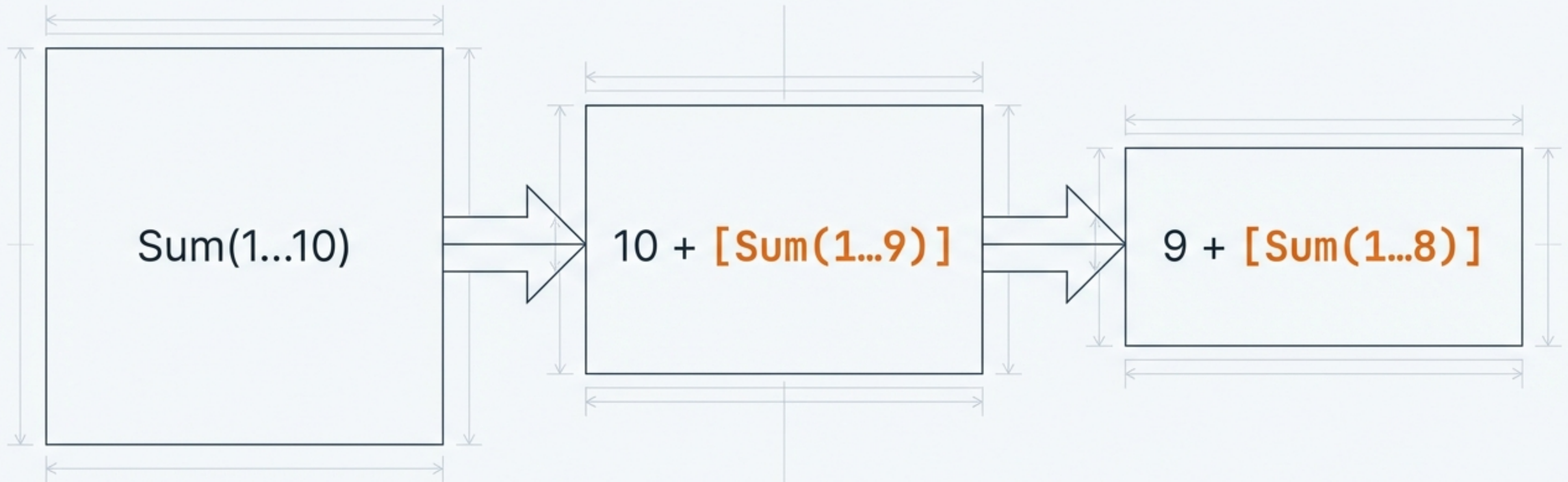
Inter Tight



- Function calls itself.
- Code is cleaner and shorter.
- Essential for Trees and Graphs.

Any iterative code can be converted to recursive code.
The choice depends on elegance and complexity.

The Recursive Strategy: Divide and Conquer



We break the problem down until it becomes so small that the answer is obvious.

Anatomy of a Recursive Function

1. The Base Case
(The Anchor)

```
def recursive_function(n):  
    → if n == 1:  
        return 1
```

2. The Logic

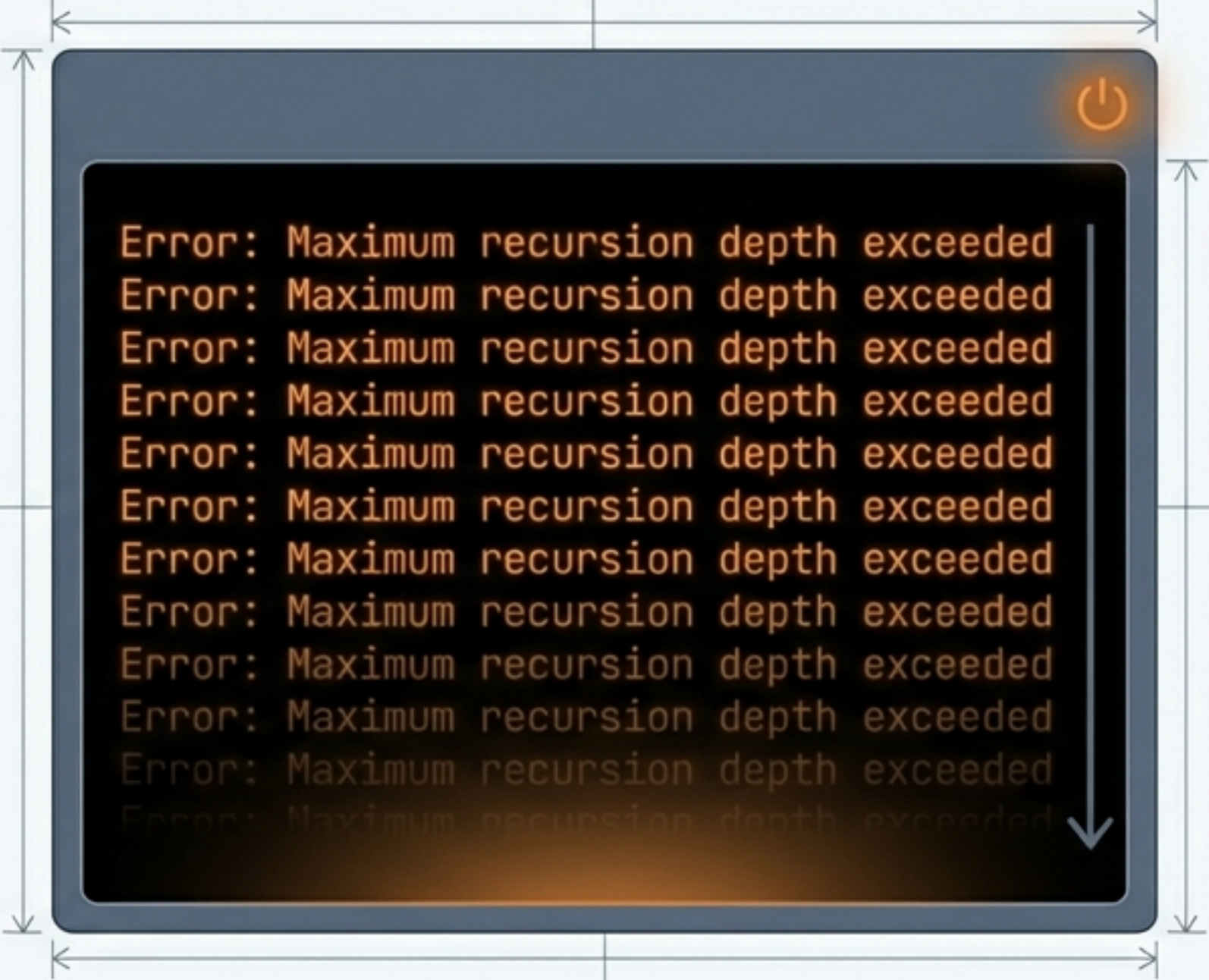
```
    # Do some work ←
```

```
    return n * recursive_function(n-1)
```

3. The Recursive Call (The Engine)

The Danger Zone: Infinite Recursion

- If you forget the Base Case, the function never stops.
- The program runs until the computer's stack memory is full, causing a RecursionError.
- **Rule:** Always define WHEN to stop before defining WHAT to do.



```
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
Error: Maximum recursion depth exceeded
```


Case Study 1: The Factorial (n!)

$$5! = 5 * 4 * 3 * 2 * 1 = 120$$

Linear Recursion

$$n! = n * (n-1)!$$

Trust the Process: To find 5!,
assume we can find 4!.

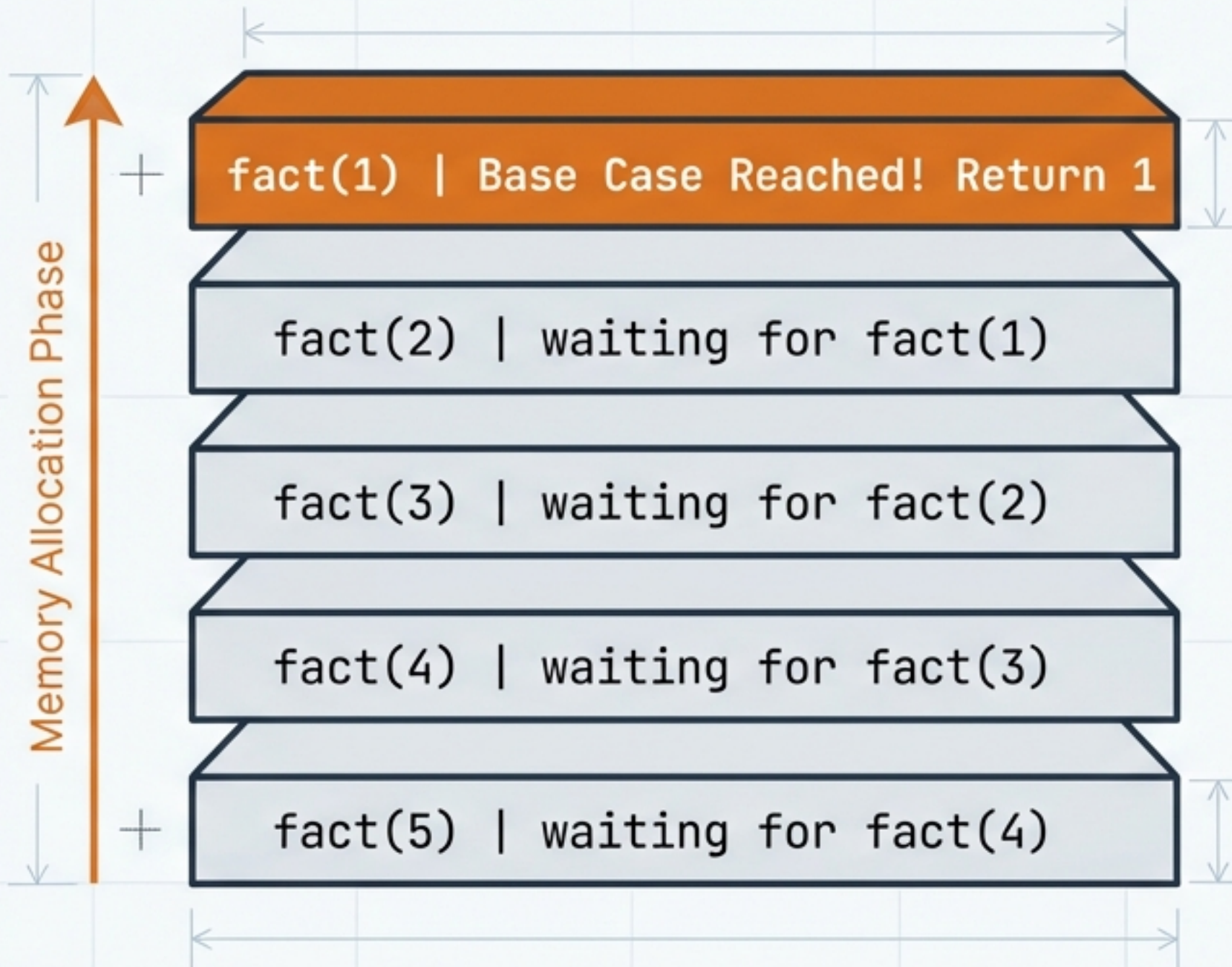
Factorial: The Code

```
def factorial(n):  
    # Base Case: Stop at 0 or 1  
    if n == 0 or n == 1:  
        return 1  
  
    # Recursive Step  
    return n * factorial(n-1)
```

**Terminating
Condition**

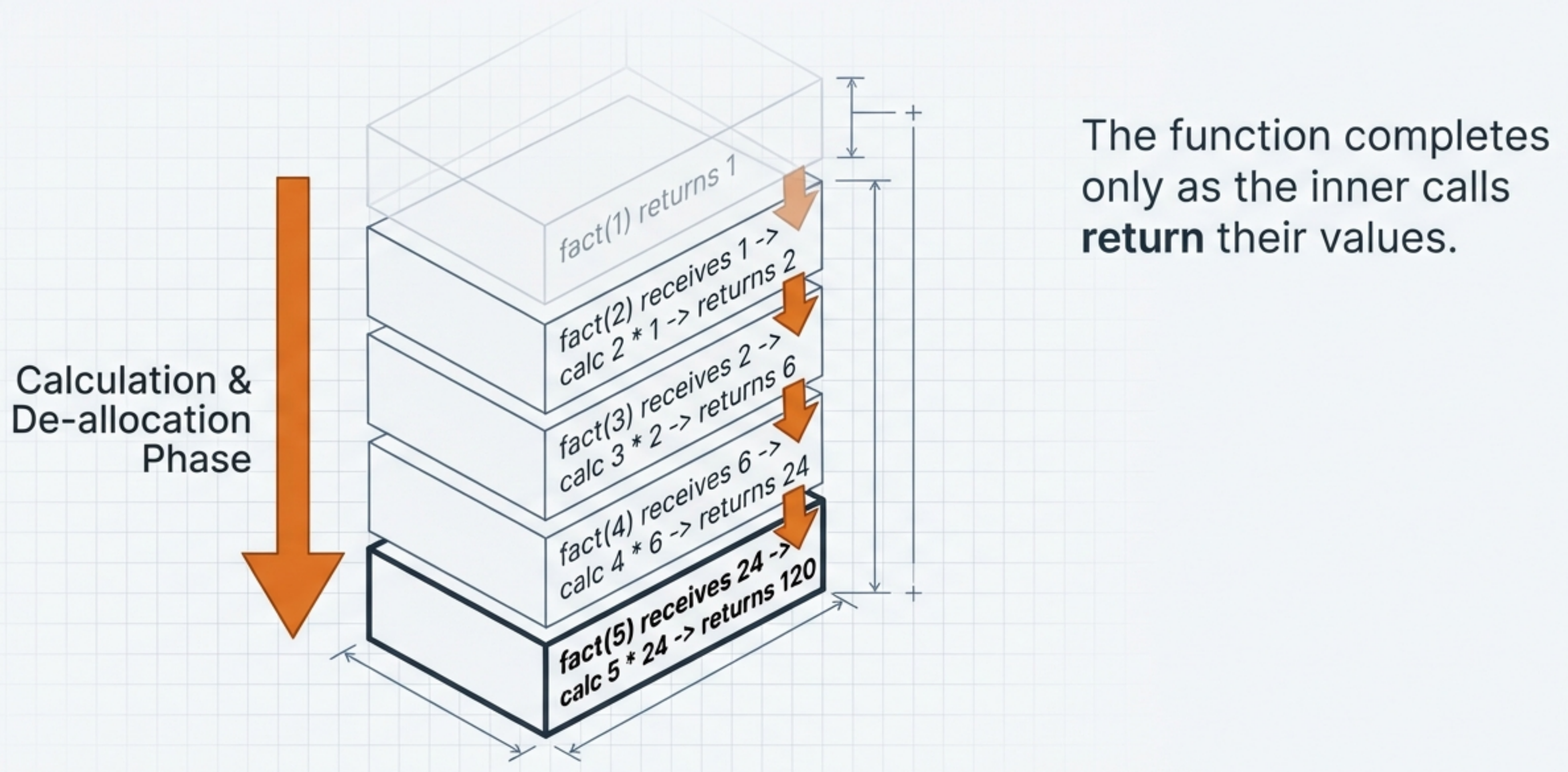
**Recursive
Step**

Visualizing the “Dive” (Stack Allocation)



When fact(5) calls fact(4), it pauses. A new memory block is created. Nothing is calculated yet.

Visualizing the "Return" (Stack Unwinding)



Case Study 2: Fibonacci Series

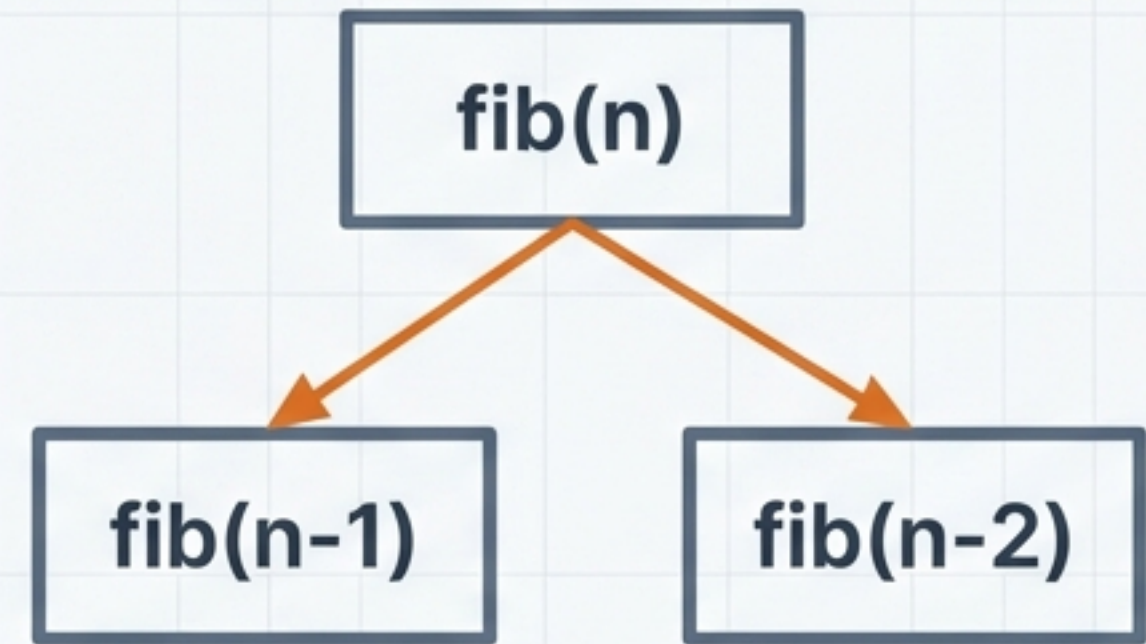
Logic: Each number is the sum of the two preceding ones.

0, 1, 1, 2, 3, 5, 8, 13, 21...

A sequence of numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21... Below the numbers, orange curved arrows point from each number to the next one. Small orange plus signs are placed below the arrows between the 1st and 2nd, 2nd and 3rd, 3rd and 4th, 4th and 5th, 5th and 6th, 6th and 7th, and 7th and 8th numbers, indicating that each number is the sum of the two preceding ones.

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

One call triggers TWO calls.



Branching Recursion:
The path splits.

Fibonacci: The Code

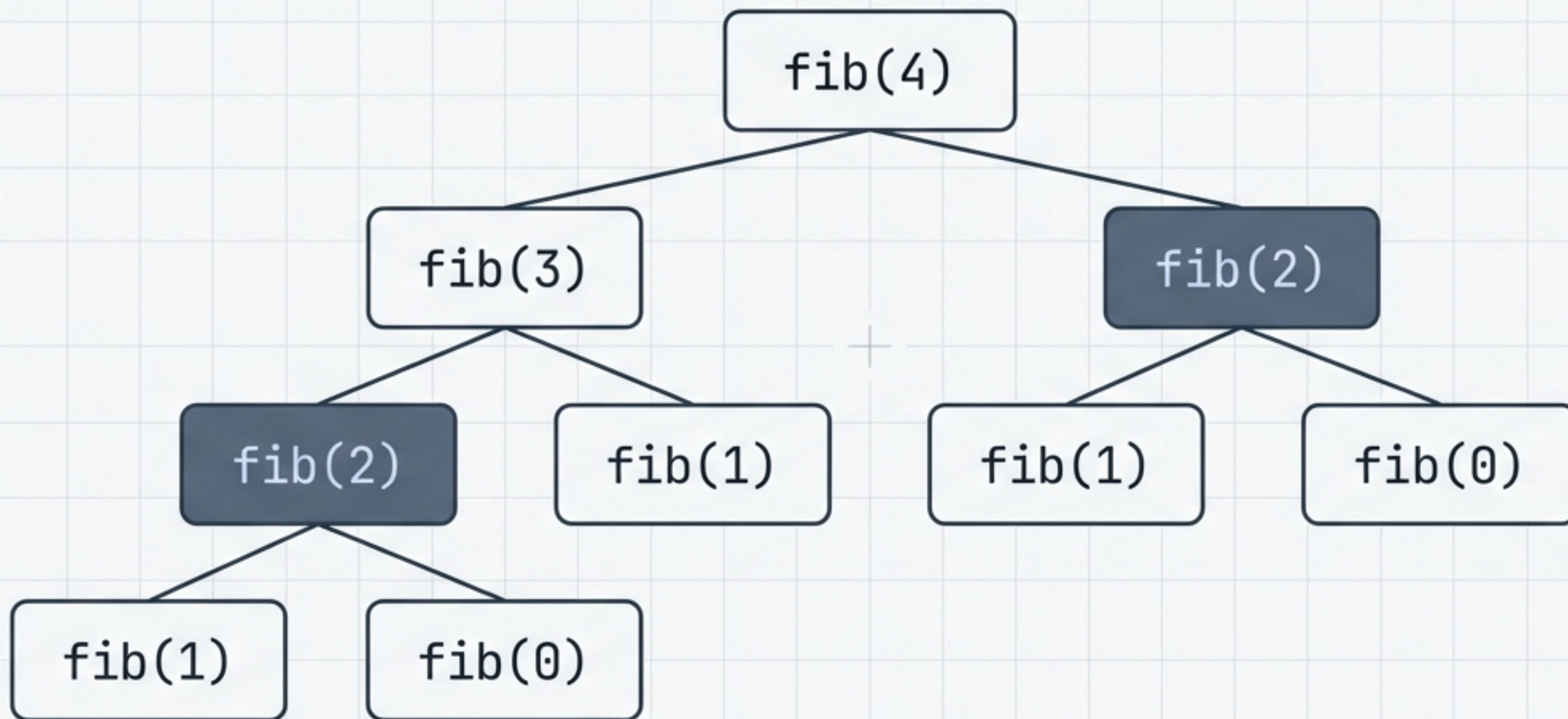
```
def fib(n):  
    # Base Case  
    if n == 1 or n == 2:  
        return 1
```

```
    # Recursive Step  
    return fib(n-1) + fib(n-2)
```

Double Call. The computer must finish the entire left side (n-1) before it touches the right side (n-2).



The Execution Tree



Visualizing Exponential Growth. Note how 'fib(2)' is calculated multiple times.

How to 'Dry Run' Recursive Code



Write Variables

List out the initial variables and their values.



Draw the Box

Create a visual container for the current function execution.



Check Base Case

Determine if the condition to stop recursion is met.



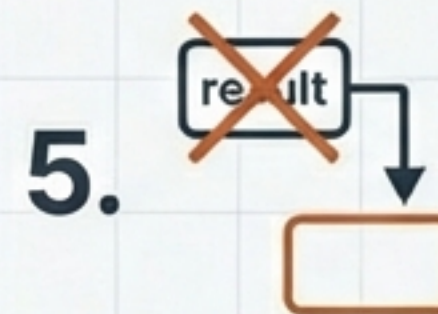
Check Base Case

Determine if the condition to stop recursion is met.



Pause & Stack

If a recursive call is made, pause current box, draw new one on top.

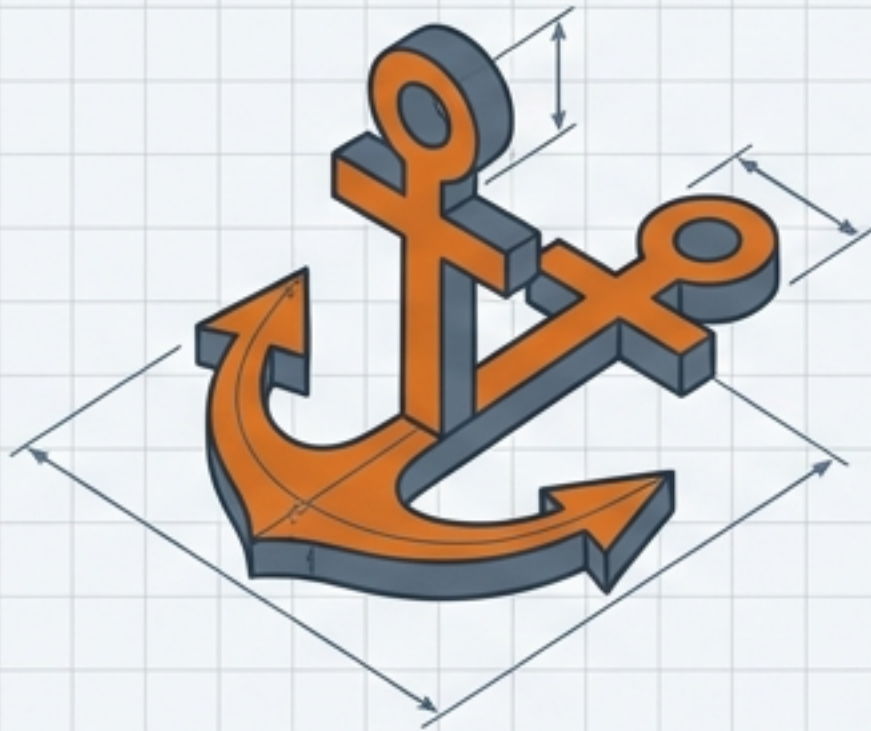


Unwind

When a box returns, cross it out and pass value down.

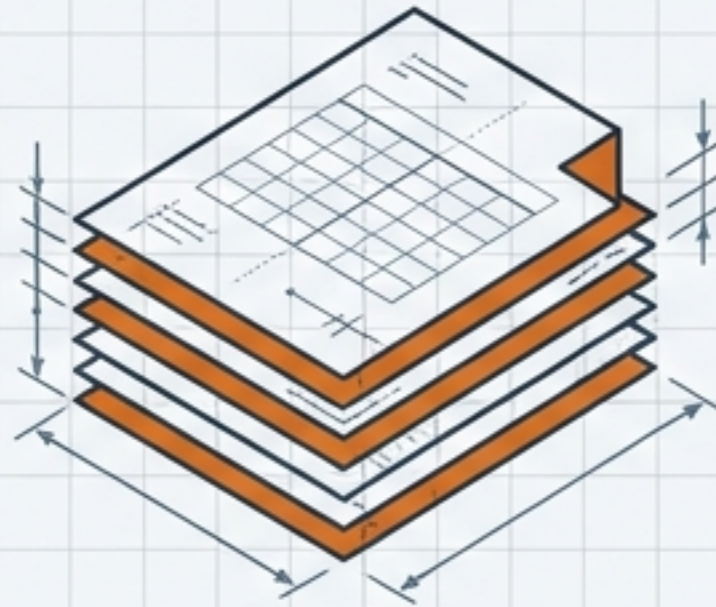
Don't just stare at the code. Simulate the machine.

Summary & The Road Ahead



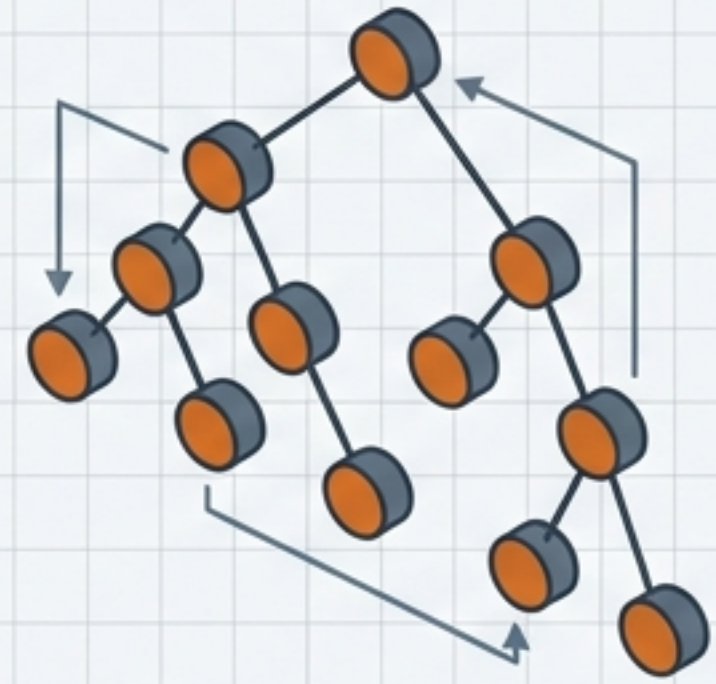
The Base Case

Non-negotiable. Defines when to stop.



The Stack

Trust the function to solve the smaller problem.



Next Topic

Recursion is the key to understanding Trees and Graphs.

Practice tracing. Grab a pen and paper.